Computación de Alto Rendimiento (HPC)

Caso de Estudio 2 — Concurrencia con OpenMP,
Perfilado y Benchmark

Paralelización con OpenMP de la multiplicación de matrices,

Caracterización de CPU y memoria, y
comparación de rendimiento
entre dos máquinas

José Alejandro Quintero Gahona

Meycol Kleym Quintero Castaño

María Angélica Alvarez Giraldo

Docente: Ramiro Andrés Barrios Valencia

Asignatura: Computación de Alto Rendimiento

1 de noviembre de 2026

Índice

1. Introducción y objetivos **3**

2. Conceptos mínimos **3**
- 2.1. OpenMP . . . 3
- 2.2. Métricas de rendimiento . . . 4
- 2.3. Benchmark y perfilado . . . 4
- 2.4. Caché y miss rate . . . 5

3. El kernel y cómo se mide **5**
- 3.1. El algoritmo . . . 5
- 3.2. El patrón de acceso a memoria . . . 6
- 3.3. Medición del tiempo . . . 6
- 3.4. Por qué 10 corridas . . . 6

4. Implementación con OpenMP **6**
- 4.1. La directiva . . . 6
- 4.2. Justificación a partir de los ejemplos de la especificación . . . 7
- 4.3. Por qué cada elemento . . . 7
- 4.4. Por qué no se necesitan otras cláusulas . . . 8
- 4.5. Configuración del número de hilos . . . 8
- 4.6. El flag de compilación no es opcional . . . 8

5. Diseño experimental: perfilado y benchmark **8**
- 5.1. Qué se mide en cada parte . . . 8
- 5.2. Variables . . . 9
- 5.3. Protocolo de comparabilidad . . . 9
- 5.4. Validación de correctitud . . . 9
- 5.5. Herramientas . . . 10

6. Resultados **10**
- 6.1. Perfilado de CPU: dónde se va el tiempo . . . 10
- 6.2. Perfilado de memoria: el comportamiento de la caché . . . 10
- 6.3. Desempeño: las cinco estrategias comparadas . . . 11
- 6.4. Escalamiento de OpenMP . . . 11
- 6.5. Benchmark entre las dos máquinas . . . 12

7. Análisis de resultados **12**
- 7.1. El perfilado confirma la hipótesis del caso de estudio 1 . . . 12
- 7.2. Por qué OpenMP no supera al kernel transpuesto . . . 13
- 7.3. Por qué el speedup crece con el tamaño de N . . . 14
- 7.4. Qué nos enseña la comparación entre las dos máquinas . . . 14
- 7.5. Granularidad del reparto contra localidad de datos . . . 15
- 7.6. El error metodológico que casi distorsionó el resultado . . . 15

8. Conclusiones **16**
8.1. Hallazgos 16
8.2. Limitaciones 16
8.3. Trabajo futuro 17

9. Referencias **17**

1. Introducción y objetivos

Qué se hizo

Este documento presenta el caso de estudio 2 de Computación de Alto Rendimiento (HPC). Parte del programa secuencial de multiplicación de matrices del caso de estudio 1 y, sobre él, implementa tres cosas que el enunciado pide:

1. **Concurrencia con OpenMP**, con las directivas elegidas y justificadas a partir de los ejemplos de la especificación oficial.
2. **Perfilado de la aplicación** en sus dos ejes mínimos: CPU y memoria.
3. **Benchmark en dos máquinas distintas**, con hardware realmente diferente (Intel vs AMD, distinta jerarquía de caché, distinto sistema operativo).

El caso de estudio 1 ya había comparado cuatro estrategias: pthreads, `fork()` con memoria compartida, optimización de compilación (`-O3` y variantes) y un kernel transpuesto. Aquí se completa el cuadro con OpenMP como quinta estrategia y, sobre todo, se responde con **medición** una pregunta que antes era intuición: *¿por qué la transposición de B hace que el kernel sea tan más rápido?*

Objetivos

- Implementar la versión concurrente con OpenMP y justificar técnicamente cada directiva y cláusula utilizada.
- Caracterizar la aplicación: dónde se va el tiempo de CPU y qué presión genera sobre la memoria.
- Medir el rendimiento en dos máquinas Linux distintas y comparar los resultados.
- Integrar las tres mediciones (perfilado, speedup y benchmark) en una explicación única del comportamiento del programa.

Organización

Las secciones 2 y 3 presentan los conceptos y el programa base. La sección 4 describe la implementación con OpenMP. La sección 5 expone el diseño experimental. La sección 6 muestra los resultados, la 7 los analiza y la 8 cierra con las conclusiones.

2. Conceptos mínimos

2.1. OpenMP

OpenMP es una **API de paralelización para arquitecturas de memoria compartida**. No es un lenguaje: es un conjunto de **directivas de compilador** (centinela `#pragma omp`), **rutinas de biblioteca** y **variables de ambiente**.

La **región paralela** es el bloque central: el hilo maestro crea un equipo de hilos, todos ejecutan el bloque, y al final hay una barrera implícita que espera a que todos terminen. Un **loop construct** reparte las iteraciones de un bucle entre los hilos del equipo.

Dos detalles que importan para este trabajo:

- Las directivas se **ignoran silenciosamente** si el programa no se compila con `-fopenmp`. El flag no es opcional.
- Por defecto las variables son *compartidas* entre los hilos, salvo las variables de control de los bucles, que son *privadas*. Se puede forzar con las cláusulas `shared()` y `private()`.

2.2. Métricas de rendimiento

Hay dos formas de medir el rendimiento computacional: el **tiempo de respuesta** (cuánto tarda *una* aplicación, la métrica del usuario final) y el **throughput** (cuánto trabajo completa el sistema por unidad de tiempo, la métrica del sistema).

Para comparar dos máquinas, el rendimiento se define como $R = 1/T$ y la diferencia porcentual como

$$\%\Delta = \frac{R_x - R_y}{R_y} \times 100$$

Las unidades históricas son los **MIPS** (millones de instrucciones por segundo), que dependen del repertorio de instrucciones y por ello no son comparables entre arquitecturas, y los **FLOPS** (operaciones de coma flotante por segundo), que son la unidad moderna pero **solo aplican a operaciones de coma flotante**.

Como el kernel de este trabajo trabaja con **enteros**, **no se puede reportar en FLOPS**. Se usan en su lugar el tiempo absoluto y los **GOP/s** (*giga integer operations per second*), análogo entero: el kernel hace N^3 multiplicaciones y N^3 sumas, así que se cuentan $2N^3$ operaciones por segundo.

2.3. Benchmark y perfilado

Un **benchmark** es un programa que genera una *carga de trabajo* conocida para evaluar el rendimiento de un equipo. Aplicado a un caso real, lo correcto es medir el **kernel**: la parte central del programa que consume la mayor parte del tiempo. Los benchmarks reducidos o sintéticos no son una buena opción para caracterizar una aplicación real, porque no generan una carga realista sobre la memoria.

El **perfilado** tiene otro objetivo: no medir el desempeño sino **identificar el comportamiento del programa** en una arquitectura concreta, es decir, dónde se inefficiencies, dónde hay embotellamiento de memoria, y qué explica el escalamiento. Hay dos familias de herramientas que se usan aquí:

- **GProf**: perfilador por muestreo integrado con el compilador. Se activa con la bandera `-pg`, que inserta instrumentación; el programa escribe un archivo `gmon.out` al ejecutarse, y la utilidad `gprof` lo convierte en un reporte legible.

- **Cachegrind**: simulador de caché del proyecto Valgrind. Reproduce el comportamiento de una jerarquía de caché durante la ejecución y reporta, por función, las instrucciones ejecutadas y los **fallos de caché** (*misses*) en L1 y en el último nivel. Con la opción `-cache-sim=yes` añade las tasas de fallo de lectura y escritura.

Un tercer dato, el **max RSS** (*maximum resident set size*), da el tamaño del espacio de memoria utilizado y se obtiene con la llamada al sistema `getrusage`, sin depender de ninguna herramienta externa.

2.4. Caché y miss rate

El procesador no accede a la RAM directamente: hay una jerarquía de cachés, pequeñas y muy rápidas, entre el núcleo y la memoria. El **miss rate** es la fracción de accesos a un nivel que no se encuentran en ese nivel y deben bajar al siguiente. Un miss a RAM cuesta cientos de ciclos, mientras que un acierto a L1 cuesta unos pocos, por lo que el miss rate explica directamente el rendimiento de un kernel.

Cuando se dice que un acceso es *contiguo* se está describiendo el patrón con que se recorre un arreglo en memoria. En el kernel clásico de multiplicación de matrices el acceso a B **no** es contiguo, y esa es la raíz del problema que se mide en la sección 6.

3. El kernel y cómo se mide

3.1. El algoritmo

El programa calcula $C = A \times B$ con el algoritmo clásico, donde

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \cdot B[k][j]$$

Son N^2 elementos a producir y N productos cada uno: la complejidad es $O(N^3)$. El bucle es el mismo que después se paraleliza:

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        for (int k = 0; k < N; k++)
            C[i*N + j] += A[i*N + k] * B[k*N + j];
```

Los índices `i*N + k` y `k*N + j` convierten el índice bidimensional en uno unidimensional porque las matrices viven en un bloque contiguo de memoria (allocado con `malloc`).

El código está modularizado: `modules/memoria.c` reserva, `modules/llenado.c` genera los datos y `modules/tiempo_mult.c` contiene el kernel y la medición. La versión con OpenMP es un módulo aparte (`modules/tiempo_mult_openmp.c`) con su propio header, lo que permite medir el *kernel* de forma aislada, que es lo que interesa para el perfilado.

3.2. El patrón de acceso a memoria

Este detalle es central para el análisis, así que conviene hacerlo explícito desde el inicio. En el bucle interior, con i y j fijos, el índice k recorre:

- $A[i \cdot N + k]$: acceso **contiguo**. Al avanzar k, la dirección avanza de a un entero; la fila i de A se recorre secuencialmente.
- $B[k \cdot N + j]$: acceso **con salto**. Al avanzar k, la dirección avanza N enteros; la columna j de B se recorre a saltos y cada salto cae normalmente en una línea de caché diferente.
- $C[i \cdot N + j]$: acceso **constante**; queda en un registro.

Es decir: la mitad del trabajo de la multiplicación (la lectura de B) accede a la memoria en un orden que no corresponde al orden en que los datos están almacenados. Este patrón es el que mide Cachegrind en la sección 6 y el que explica, en el análisis, por qué la transposición de B produce una ganancia tan grande sin necesidad de paralelizar.

3.3. Medición del tiempo

Se mide el **wall clock** con `clock_gettime(CLOCK_MONOTONIC_RAW, ...)`: es monotónica (nunca retrocede) y *raw*, es decir, inmune a los ajustes que NTP hace sobre el reloj del sistema, que meterían ruido en las mediciones cortas. La alternativa `clock()` mide tiempo de CPU, que es otra cosa.

Se cronometra **solo la multiplicación**: la reserva de memoria, el llenado y la impresión quedan fuera, porque no son el trabajo analizado. El perfilado de GProf confirma esta decisión: el 100 % del tiempo propio está en la función del kernel.

3.4. Por qué 10 corridas

Una sola medición no es confiable: el sistema puede tener otra tarea corriendo, una preempción puede caer en la corrida, o el proceso puede usar la CPU en el momento crítico. Se repiten **10 corridas por configuración** y se toma la **mediana**, que es robusta frente a valores atípicos. Además, el script recorre las repeticiones por fuera y los tamaños por dentro: si las 10 repeticiones de un mismo N fueran seguidas, los datos quedarían *calientes* en la caché y la medición saldría artificialmente rápida.

4. Implementación con OpenMP

4.1. La directiva

El pragma que se aplica al triple bucle, en `modules/tiempo_mult_openmp.c`, es:

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
```

```
        for (int k = 0; k < N; k++) {
            C[(size_t)i * N + j] +=
                A[(size_t)i * N + k] *
                B[(size_t)k * N + j];
        }
    }
}
```

4.2. Justificación a partir de los ejemplos de la especificación

Antes de escribir el código se revisaron los ejemplos del documento oficial *OpenMP Application Programming Interface Examples v6.0.1* (`github.com/OpenMP/Examples`), escogiendo las directivas entendiendo el porqué y no al azar. Del índice, tres ejemplos aplican al patrón del kernel:

Ejemplo	**Patrón**	**Veredicto**
3.1 `ploop.1.c`	`#pragma omp parallel for` sobre un solo loop	Solo paraleliza el loop externo i. Con $N = 4000$ son 4000 bloques, mal repartidos entre 12 núcleos.
3.8 `collapse.1.c`	`#pragma omp for collapse(2)` sobre dos loops anidados	**Elegido.** Une i y j en un único espacio de N^2 iteraciones (16 millones para $N = 4000$) con buen reparto entre hilos.
5.3.5 `task_dep.5.c`	`#pragma omp task depend(in/inout)` sobre bloques	Descartado. Asume un kernel bloqueado y dependencias explícitas entre bloques; más complejo de lo que el caso requiere.

4.3. Por qué cada elemento

- `parallel for` **(combinada)**: crea el equipo de hilos y reparte el trabajo del loop en una sola directiva. El ejemplo 3.1 del PDF muestra las dos formas equivalentes (combinada y separada) y la combinada es más limpia.

- `collapse(2)`: es la clave de la elección. Colapsar dos loops anidados significa que el *runtime* los trata como un único espacio de iteraciones de tamaño $N \times N$ y reparte ese espacio entre los hilos. Sin `collapse` solo se paralelizaría i, y con $N = 2000$ eso daría 2000 bloques para 12 núcleos: un reparto muy desigual. El loop k queda secuencial dentro de cada par (i, j), que es lo correcto, porque la acumulación del producto punto debe permanecer en un solo hilo.

- **No se usa** `private(k)` **explícito**: en C99 las variables de control de un `for` ya son implícitamente privadas dentro de un loop construct. Declararlo además *falla la compilación*, porque k está declarada dentro del `for` y no es visible en el punto donde aparece la directiva.

4.4. Por qué no se necesitan otras cláusulas

Este es el punto donde el conocimiento del algoritmo garantiza la corrección:

- A, B y C quedan **shared** por defecto, que es lo correcto: A y B solo se leen, y cada hilo solo escribe en $C[i \cdot N + j]$ para el par (i, j) concreto que le tocó en el espacio colapsado. No hay race condition sobre C.
- **No hace falta** `reduction(+:)`. Una *reduction* se usa cuando varios hilos acumulan sobre la *misma* variable. Aquí cada $C[i][j]$ lo acumula íntegramente un solo hilo y los hilos distintos escriben en celdas distintas.
- **No hace falta** `critical` **ni** `atomic`: no existe región crítica.
- **No hace falta** `barrier` explícita: la región paralela ya tiene una barrera implícita al final.

4.5. Configuración del número de hilos

El número de hilos se puede fijar en tres lugares, por orden de precedencia: la cláusula `num_threads()` de la directiva, la llamada a `omp_set_num_threads()` y la variable de ambiente `OMP_NUM_THREADS`. Aquí se usa la variable de ambiente, que es la forma más portable y además permite barrer el número de hilos *sin recompilar* el binario.

4.6. El flag de compilación no es opcional

Un detalle que cuesta caro si se pasa por alto: **las directivas se ignoran silenciosamente si no se compila con** `-fopenmp`. El programa compilaría sin errores, se ejecutaría, y daría exactamente el mismo resultado secuencial.

Además, el flag de optimización debe ser **el mismo que usan las demás estrategias** de la comparación. La primera versión de este trabajo compiló el binario de OpenMP con `-O0` mientras el resto usaba `-O2`; esa diferencia hacía que el speedup de OpenMP pareciera artificialmente malo (entre $0.71\times$ y $1.48\times$) porque la comparación mezclaba dos efectos distintos: el paralelismo y la ausencia de optimización del compilador. Corregido a `-fopenmp -O2`, el $T = 1$ de OpenMP da 76.02 s contra los 75.65 s del secuencial de referencia (0.5 % de diferencia), lo que confirma que la base de comparación quedó justa.

5. Diseño experimental: perfilado y benchmark

5.1. Qué se mide en cada parte

El **perfilado** caracteriza la aplicación: dice dónde se va el tiempo de CPU y cuánta presión genera sobre la memoria. El perfilado de CPU (GProf) se corre en las dos máquinas; el detalle fino de caché (Cachegrind) se midió en la máquina A, que es donde se corrió el experimento de desempeño principal.

El **benchmark** compara las dos máquinas. Como se explicó en la sección 2, la métrica de esa comparación es el **rendimiento** ($R = 1/T$, y su análogo entero GOP/s). El **speedup** es un dato complementario que sale de las mismas corridas y dice qué equipo paraleliza mejor. Se reportan ambos porque juntos cuentan la historia completa: el rendimiento dice quién resuelve el problema más rápido, y el speedup dice por qué.

5.2. Variables

- **Tamaño de la matriz** (N): 500, 1000, 2000 y 4000.
- **Cantidad de hilos** (T): 1 (secuencial), 2, 4, 8 y 16. Es el mismo conjunto que en la rama de pthreads del caso de estudio 1, para que las curvas sean directamente comparables.
- **Variable medida**: tiempo de la multiplicación en segundos.
- **Métricas calculadas**: speedup contra el secuencial, tasa de fallo de caché, número de instrucciones ejecutadas y memoria máxima residente.

Son **200 mediciones de desempeño por máquina** (5 valores de $T \times 4$ de $N \times 10$ repeticiones), más las 5 corridas de Cachegrind y las 5 de `getrusage` del perfilado.

5.3. Protocolo de comparabilidad

Un benchmark entre máquinas solo es válido si las condiciones son equivalentes:

- **Mismo código fuente**: la misma rama del repositorio, con el mismo hash de commit.
- **Mismos flags de compilación**: `-Wall -O2 -fopenmp` en ambos equipos, con el mismo compilador.
- **Mismos datos**: semilla 42 y valor máximo 9 en todas las corridas, así que las matrices son idénticas bit a bit.
- **Misma metodología**: 10 corridas por configuración, intercalando los tamaños de matriz, y mediana como estimador.
- **Mismo estado del equipo**: sin otras cargas, conectado a la corriente (para que el procesador no baje su frecuencia) y con temperatura estable.

5.4. Validación de correctitud

Medir tiempos sin verificar que el programa calcula lo correcto es un error metodológico grave. Antes de cada corrida de medición, el script `experiments/medir_openmp.sh` ejecuta una **validación de correctitud**: genera con `experiments/verify_ref` una matriz resultado de referencia con $N = 10$ y semilla 42, ejecuta el binario de OpenMP con los mismos parámetros, y compara las dos matrices C elemento a elemento. Si **no coinciden exactamente**, el script aborta y no se mide nada. Es el mismo mecanismo usado en los labs 1 y 2 del caso de estudio 1, por lo que la implementación de referencia se reutilizó sin modificaciones.

5.5. Herramientas

Herramienta	Uso
`gcc` 14.2.0	Compilador. Flags `-Wall -O2 -fopenmp`.
`gprof`	Perfilado de CPU por muestreo con instrumentación `-pg`.
`valgrind/cachegrind`	Perfilado de memoria: simulación de caché, D1/LL misses.
`getrusage`	Memoria máxima residente (max RSS) del proceso hijo.
`lscpu`, `free`, `uname`	Captura automática del hardware de cada equipo.
`matplotlib`	Generación de las gráficas.

Valgrind no venía instalado, pero no hizo falta `sudo`: el script `experiments/bootstrap_tools.sh` descarga el paquete con `apt-get download` (que no requiere privilegios) y lo extrae con `dpkg-deb -x` en una carpeta local del proyecto, ajustando `VALGRIND_LIB` para que encuentre sus archivos de soporte.

6. Resultados

Todos los tiempos están en segundos y, cuando corresponde, se reporta la **mediana** de las 10 corridas.

6.1. Perfilado de CPU: dónde se va el tiempo

El reporte de GProf para el kernel secuencial es concluyente:

Función	% tiempo propio	llamadas	ms/llamada
`multiplicar_y_medir`	100.13	1	140.19
`llenar_matrices_con_semilla`	0.00	1	0.00
`reservar_matrices`	0.00	1	0.00

El 100 % del tiempo propio está en la función del kernel, y el llenado y la reserva de memoria son marginales. Esto confirma la decisión de cronometrar solo la multiplicación. El reporte de la máquina B es idéntico en este punto, así que la observación generaliza a hardware de otro fabricante.

6.2. Perfilado de memoria: el comportamiento de la caché

Esta es la medición más relevante del trabajo, porque cuantifica la hipótesis que el caso de estudio 1 dejó como intuición. La tabla 1 muestra los contadores de Cachegrind para las tres variantes del kernel, con $N = 500$, en la máquina A.

Tres observaciones se desprenden directamente:

Variante	T	I refs	D refs	D1 misses	D1 miss rate (lect.)
Secuencial (naive)	1	1 031 897 709	389 051 874	58 050 493	**22.4 %**
Transpuesto	1	907 904 738	389 301 885	7 893 064	**3.0 %**
OpenMP	1	1 034 704 330	389 325 981	59 227 926	22.8 %
OpenMP	8	1 062 467 457	393 297 061	59 226 557	22.5 %
OpenMP	16	1 034 768 697	389 344 956	59 231 589	22.8 %

Cuadro 1. Métricas de caché medidas con Cachegrind ($N = 500$, máquina A). D1 miss rate corresponde a las lecturas de datos.

1. **La transposición de B reduce los fallos de caché L1 en un factor 7.4**: de 58.05 millones a 7.89 millones, y la tasa de fallo de lectura baja de 22.4 % a 3.0 %. Es la medición experimental de la causa del buen rendimiento del kernel transpuesto.

2. **El número de hilos no cambia la tasa de fallo**: OpenMP con $T = 1$, 8 y 16 da 22.8 %, 22.5 % y 22.8 %. La tasa de fallo es una propiedad del *patrón de acceso*, no del paralelismo: el kernel con OpenMP es el mismo kernel naive, con el mismo acceso a B por columna.

3. **La transposición además reduce las instrucciones ejecutadas** en un 12 % (de 1 032 millones a 908 millones), porque el acceso contiguo a B es más amigable con el planificador del compilador.

El **footprint de memoria** (max RSS) es prácticamente idéntico en todas las variantes: 4.3–4.7 MB, que corresponde a las tres matrices de $N \times N$ enteros más el *overhead* de las pilas de hilo. La memoria que consume el algoritmo depende solo del tamaño de las matrices, no del hardware ni del paralelismo.

6.3. Desempeño: las cinco estrategias comparadas

Para OpenMP se reporta el mejor T por N, con la misma metodología que se aplicó a pthreads y a fork.

N	T_seq	T_pth	T_fork	T_full	T_mem	T_omp	s_pth	s_fork	s_full	s_mem	s_omp
500	0.0440	0.0315 (16)	0.0152 (8)	0.0459	0.0377	0.0160 (16)	1.40	2.89	0.96	1.17	**2.75**
1000	0.4230	0.2043 (16)	0.1169 (16)	0.4301	0.3019	0.1179 (16)	2.07	3.62	0.98	1.40	**3.59**
2000	4.3626	1.6430 (16)	1.9805 (8)	5.8909	2.6586	1.2786 (16)	2.66	2.20	0.74	1.64	**3.41**
4000	75.6511	24.2927 (16)	59.1421 (8)	70.0332	21.3223	16.6689 (8)	3.11	1.28	1.08	3.55	**4.54**

Cuadro 2. Tiempos (s, mediana de 10 corridas) y speedups de las cinco estrategias. La base es el secuencial con $-O2$. Entre paréntesis, el mejor T de cada estrategia paralela.

6.4. Escalamiento de OpenMP

Con $T = 16$ y $N = 4000$ el tiempo *empeora* frente a $T = 8$ (18.02 s contra 16.67 s): la máquina tiene 12 núcleos lógicos, de modo que 16 hilos obligan al sistema a planificar más trabajo del que puede ejecutar en paralelo, y ese *overhead* se come la ganancia.

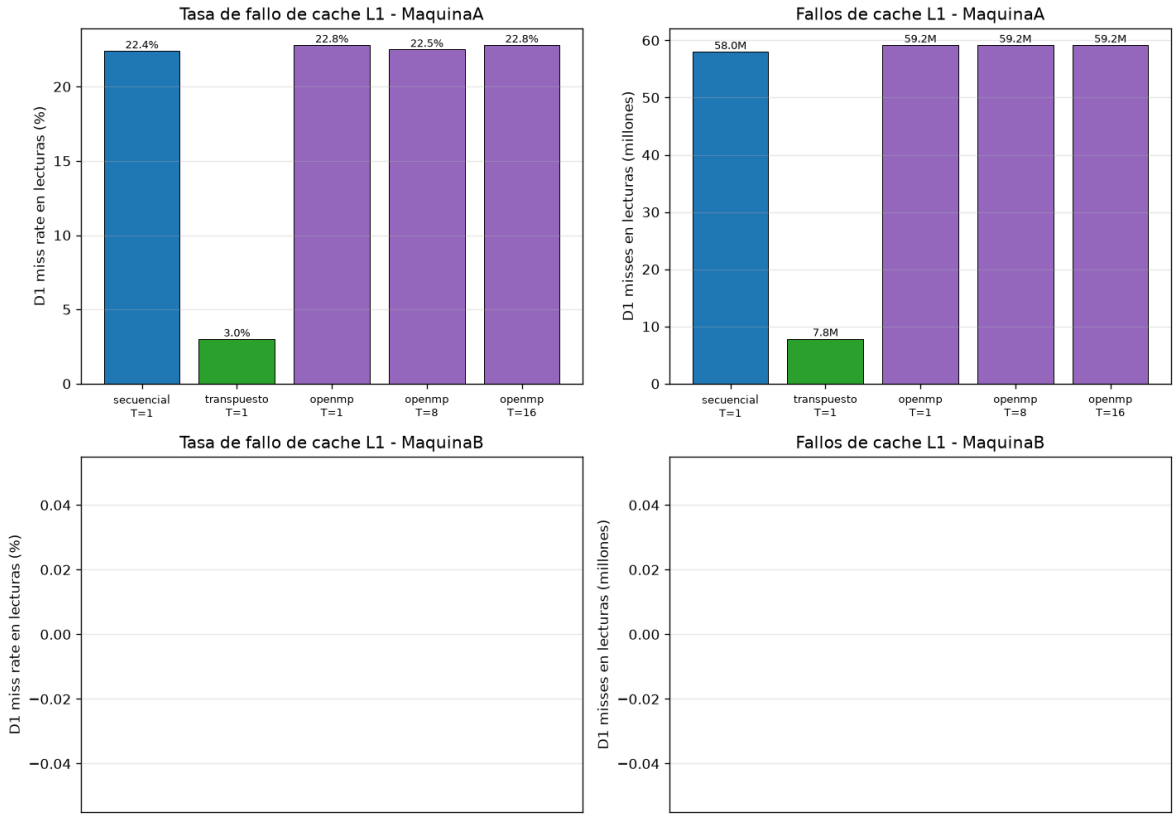


Figura 1. Tasa de fallo de caché L1 en lecturas (izquierda) y cantidad absoluta de fallos (derecha) por variante del kernel. El transpuesto (verde) reduce la tasa de 22.4 % a 3.0 %.

6.5. Benchmark entre las dos máquinas

Las dos máquinas son hardware realmente distinto: distinto fabricante, distinta arquitectura, distinta jerarquía de caché y distinto sistema operativo.

7. Análisis de resultados

7.1. El perfilado confirma la hipótesis del caso de estudio 1

En el caso de estudio 1 se observó que la transposición de B rendía mucho mejor que el kernel original y se propuso que la causa era el patrón de acceso a memoria. El perfilado lo **confirma con medición directa**: la tasa de fallo de caché L1 en lecturas baja de 22.4 % a 3.0 % al transponer B, y el número absoluto de fallos se reduce 7.4 veces. Esto no es un detalle menor: es la diferencia entre una explicación "probablemente es la caché" y una respaldada por contadores.

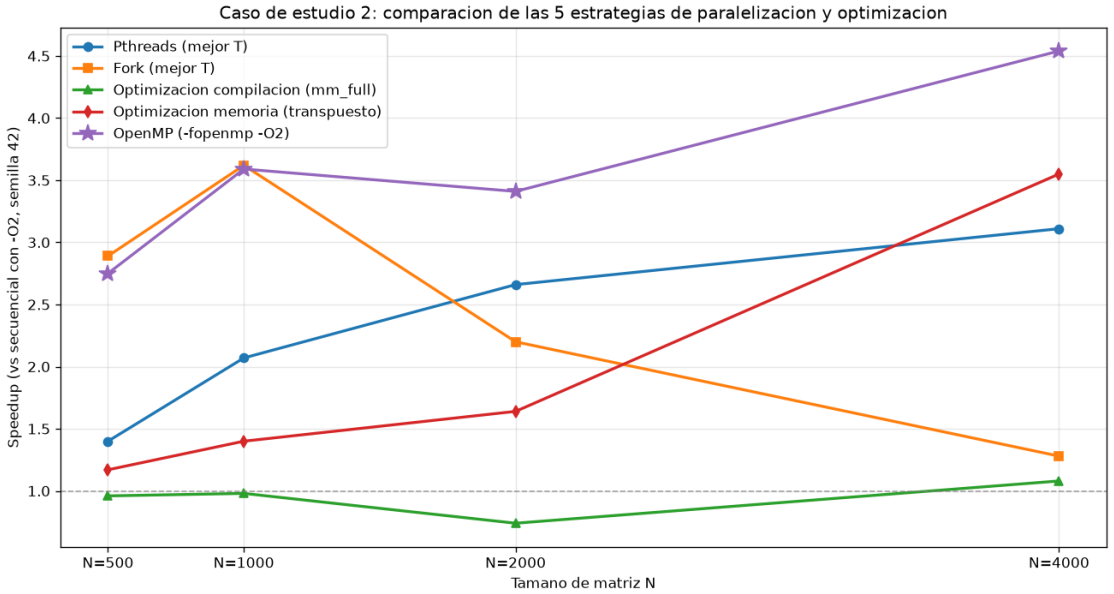


Figura 2. Speedup de las cinco estrategias frente al secuencial. OpenMP (estrella morada) es la estrategia más rápida en tres de los cuatro tamaños.

N	T=1	T=2	T=4	T=8	T=16	Speedup
500	0.0450	0.0276	0.0239	0.0180	**0.0160**	2.81×
1000	0.4257	0.2385	0.1652	0.1473	**0.1179**	3.61×
2000	4.3026	2.7034	1.5880	1.4391	**1.2786**	3.36×
4000	76.0226	34.3422	18.0106	**16.6689**	18.0214	4.56×

Cuadro 3. Tiempos (s) de OpenMP por cantidad de hilos y speedup resultante respecto al $T = 1$ de la misma máquina.

7.2. Por qué OpenMP no supera al kernel transpuesto

Los resultados de desempeño y de perfilado se combinan para explicar un fenómeno que a primera vista es sorprendente: **OpenMP con 12 hilos alcanza 4.54×, pero el kernel transpuesto *sin paralelizar nada* alcanza 3.55×**.

La explicación está en la tabla 1. El kernel con OpenMP *hereda* el mal patrón de acceso del kernel naive: la tasa de fallo de caché se mantiene en 22.5–22.8 % sin importar cuántos hilos trabajen. Añadir hilos no arregla un acceso ineficiente a la memoria; lo que hace es *multiplicar* la demanda sobre la jerarquía de caché. Con 12 hilos compitiendo por el L3 compartido de 12 MiB, la presión sobre la caché crece justo cuando el problema ya estaba limitado por ella.

El kernel transpuesto, en cambio, *reduce* la demanda de caché en un factor 7.4. Es una mejora que aplica a *cada* acceso, sin importar cuántos hilos haya. Por eso es tan competitiva: mejora el término que domina el tiempo.

Esto explica también por qué el speedup de OpenMP *no es lineal*: si el cuello de botella fuera solo el cómputo, con 8 hilos sobre 12 núcleos se esperaría un speedup cercano a 8×, y se obtuvo 4.56×. La diferencia es la contención por memoria.

Característica	Máquina A	Máquina B
CPU	Intel Core i5-12450HX (Alder Lake)	AMD Ryzen 5 5500U (Zen 2)
Núcleos físicos / lógicos	8 / 12	6 / 12
Caché L1d / L2 / L3	320 KiB / 7 MiB / 12 MiB	192 KiB / 3 MiB / 8 MiB
Memoria RAM	15 GiB	11 GiB
Sistema operativo	Debian GNU/Linux 13	Fedora Linux 41
Compilador	gcc 14.2.0	gcc 14.2.1

Cuadro 4. Hardware de las dos máquinas del benchmark.

N	T_A (s)	T_B (s)	GOP/s A	GOP/s B	%Δ	Gana rend.	s_A	s_B	Gana sp.
500	0.0160 (16)	0.0215 (16)	15.60	11.62	−25,5 %	**A**	2.81×	3.56×	**B**
1000	0.1179 (16)	0.2291 (16)	16.96	8.73	−48,5 %	**A**	3.61×	4.31×	**B**
2000	1.2786 (16)	3.2704 (8)	12.51	4.89	−60,9 %	**A**	3.36×	3.20×	**A**
4000	16.6689 (8)	59.9067 (8)	7.68	2.14	−72,2 %	**A**	4.56×	3.53×	**A**

Cuadro 5. Benchmark entre las dos máquinas. Entre paréntesis, el mejor T de hilos de cada equipo. "$\%\Delta$" es $(R_B - R_A)/R_A \times 100$ con $R = 1/T$. "Gana rend." identifica la más rápida y "Gana sp." la que mejor paraleliza.

7.3. Por qué el speedup crece con el tamaño de N

Una observación que merece explicación: el speedup de pthreads y de OpenMP *crece* a medida que N aumenta (pthreads: 1.40× en $N = 500$ hasta 3.11× en $N = 4000$; OpenMP: 2.75× hasta 4.54×). Es el comportamiento **esperado**: con N chico el trabajo total es tan pequeño que el *overhead* (crear el equipo de hilos, la barrier final, la planificación) domina el tiempo medido; con N grande el cómputo domina y el overhead se amortiza. La consecuencia es que *el speedup de una estrategia solo puede evaluarse en el contexto del tamaño del problema.*

7.4. Qué nos enseña la comparación entre las dos máquinas

El benchmark entre el Intel i5-12450HX y el AMD Ryzen 5 5500U da dos resultados complementarios.

Primero, **el hardware define la escala de forma distinta según el tamaño del problema**. La máquina B escala mejor en problemas chicos (4.31× contra 3.61× en $N = 1000$) y la máquina A escala mejor en problemas grandes (4.56× contra 3.53× en $N = 4000$). La B tiene 6 núcleos físicos homogéneos (Zen 2), mientras que la A es un diseño híbrido con núcleos de alto y de bajo rendimiento. En un problema chico el reparto de trabajo entre núcleos desiguales es ineficiente y se nota; en un problema grande el cómputo puro domina y gana el hardware más rápido. Ninguna de las dos máquinas es "mejor" en abstracto: son mejores para problemas distintos.

Segundo, **la ventaja de la máquina A se amplía con** N, de 1.34× en $N = 500$ a 3.59× en $N = 4000$. Es la firma de una diferencia de rendimiento por núcleo y por ancho de banda de memoria que el algoritmo cúbico amplifica, y es consistente con la jerarquía de caché: la A tiene 12 MiB de L3 contra 8 MiB de la B. Como el acceso a la caché es el cuello de botella de este problema, no es sorprendente que las diferencias de caché se traduzcan directamente en diferencias de tiempo.

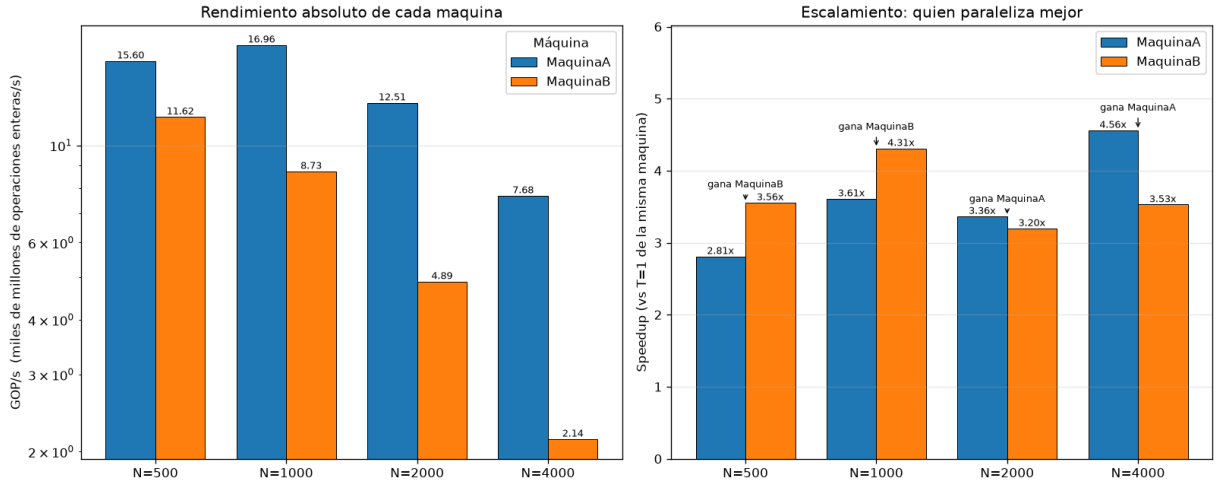


Figura 3. Benchmark entre máquinas. Izquierda: rendimiento absoluto en GOP/s; la máquina A gana en todos los tamaños. Derecha: speedup interno con el ganador anotado; la B gana en $N = 500$ y $N = 1000$, la A en $N = 2000$ y $N = 4000$.

Un detalle adicional: el T óptimo es **el mismo en ambas máquinas** para cada N (16 hasta $N = 2000$, 8 en $N = 4000$), lo que sugiere que la saturación la determina la estructura del problema y no el hardware.

7.5. Granularidad del reparto contra localidad de datos

OpenMP (4.54× en $N = 4000$) supera a pthreads manual (3.11×), pese a ser la misma idea: hilos compartiendo memoria. La diferencia está en el **reparto del trabajo**. La versión manual reparte *filas completas* de la matriz entre los hilos, de modo que cada hilo trabaja sobre un bloque contiguo de filas y conserva la localidad. La versión con `collapse(2)` reparte *pares* (i, j) del espacio de N^2 iteraciones, y pares consecutivos asignados al mismo hilo pueden estar muy lejos en memoria.

En este problema, donde el acceso a B ya es el problema, la localidad del reparto importa más que la finerza del reparto: un reparto más fino y más uniforme no compensa una peor localidad.

7.6. El error metodológico que casi distorsionó el resultado

La primera medición de OpenMP se hizo compilando el binario con `-fopenmp -O0`, mientras todas las demás estrategias usaban `-O2`. El resultado fue un speedup de entre 0.71× y 1.48×, que hacía ver a OpenMP como la peor de las cinco estrategias.

El error no era del código sino del **diseño del experimento**: al comparar un binario sin optimizar contra una base optimizada, la métrica mezclaba dos efectos distintos y la conclusión sobre el rendimiento de OpenMP era errónea. Corregido el flag, el $T = 1$ de OpenMP da 76.02 s contra los 75.65 s del secuencial (0.5 % de diferencia), lo que confirma que la comparación quedó justa, y el speedup real resultó ser el *mejor* de las cinco estrategias.

La lección es general: **comparar estrategias de paralelización exige que todas usen el mismo nivel de optimización del compilador**.

8. Conclusiones

8.1. Hallazgos

1. **El perfilado cuantifica lo que el caso de estudio 1 solo sospechaba**. Transponer B baja la tasa de fallo de caché L1 en lecturas de 22.4 % a 3.0 % (7.4× menos fallos) y reduce un 12 % las instrucciones ejecutadas. La hipótesis de que el cuello de botella era la memoria queda respaldada por medición directa.

2. **La tasa de fallo no depende del número de hilos** (22.8 % / 22.5 % / 22.8 % para $T = 1/8/16$). Eso aísla la causa: el paralelismo no empeora el acceso a la memoria, pero tampoco lo arregla.

3. **OpenMP es la estrategia más rápida de las cinco**: 4.54× en $N = 4000$, por encima del kernel transpuesto (3.55×) y de pthreads manual (3.11×). La explicación de por qué no supera al transpuesto está en el hallazgo anterior: paralelizar multiplica la demanda sobre una caché que ya estaba saturada.

4. **El speedup crece con el tamaño del problema**, como corresponde a un comportamiento sano: con problemas chicos domina el *overhead*, con problemas grandes el cómputo lo amortiza.

5. **El hardware define la escala según el tamaño del problema**. En el benchmark entre el Intel i5-12450HX y el AMD Ryzen 5 5500U, la B escala mejor en problemas chicos (6 núcleos físicos homogéneos contra la arquitectura híbrida de la A) y la A en problemas grandes. La ventaja en tiempo de la A crece con N (de 1.34× a 3.59×), en consonancia con su mayor jerarquía de caché.

6. **Comparar estrategias exige igualar las condiciones de compilación**. La medición inicial con `-O0` para OpenMP y `-O2` para el resto llevó a la conclusión errónea de que OpenMP era la peor estrategia. Es un error metodológico, no de implementación.

8.2. Limitaciones

- El perfilado con Cachegrind se ejecutó con $N = 500$ (verificado estable hasta $N = 1000$), porque la simulación de caché es entre 50 y 100 veces más lenta que la ejecución nativa. La tasa de fallo es una propiedad del patrón de acceso y no debería variar con N, pero no se verificó para los tamaños mayores.

- GProf no es concluyente para la variante con OpenMP: solo instrumenta el hilo principal, por lo que el reparto de tiempo dentro de la región paralela no se puede atribuir con esta herramienta.

- La comparación entre máquinas no pudo controlar la versión del *firmware*, la configuración de la BIOS ni el estado térmico de los equipos, factores que pueden introducir diferencias de varios puntos porcentuales.

8.3. Trabajo futuro

- **Combinar las dos optimizaciones**: aplicar la directiva de OpenMP sobre el kernel *transpuesto*. Es la hipótesis más interesante que queda abierta: la transposición reduce la demanda de caché y el paralelismo reparte el trabajo restante.
- **Reparto por bloques en la versión con OpenMP**: usar `collapse(2)` solo sobre el loop j y particionar i por bloques en el nivel externo, para recuperar la localidad que tenía la versión manual.
- **Comparar con más de dos máquinas**, para caracterizar el comportamiento del kernel en función de las características del hardware (núcleos, jerarquía de caché, ancho de banda de memoria).

9. Referencias

- **OpenMP Application Programming Interface Examples, Version 6.0.1 (noviembre 2025)** — fuente oficial de los ejemplos utilizados para justificar la elección de las directivas `parallel for collapse(2)` (secciones 3.1, 3.8 y 5.3.5). Disponible en `github.com/OpenMP/Examples` (etiqueta `v6.0.1`).
- **OpenMP Application Programming Interface Specification, Version 6.0 (noviembre 2024)** — especificación del estándar que define directivas, cláusulas, rutinas de biblioteca y variables de ambiente. Disponible en `openmp.org`.
- **GNU `gprof`** — perfilador por muestreo con instrumentación en tiempo de compilación (`-pg`). Manual en `sourceware.org/binutils/docs`.
- **Valgrind:** `cachegrind` — simulador de caché del proyecto Valgrind. Documentación en `valgrind.org/docs/manual/cg-manual.html`.
- **POSIX.1-2017 (IEEE Std 1003.1-2017)** — `clock_gettime`, `getrusage`, `sched_getaffinity`. Especificación de la API POSIX utilizada para la medición del tiempo y de la memoria residente.
- **Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 2** — referencia de los contadores de rendimiento del procesador y del modelo de jerarquía de caché.
- **Brendan Gregg** — ***Systems Performance*** y su blog `brendangregg.com` — referencias sobre las herramientas de línea de comandos para rendimiento en Linux.

- **Material del curso de Computación de Alto Rendimiento** — presentaciones y grabaciones de clase (OpenMP, benchmark, profiling) utilizadas como referencia conceptual durante el desarrollo del caso de estudio 2.